

Angular (v20+) Hands-On Lab Report

Incremental Project: Student Course Portal

Hands-On:	Hands-On 7 [Advanced-Professional]
Topic:	Advanced Routing, Dynamic Params, Guards & Lazy Loading
Developer:	Cathy George
OS Version:	Windows 11
Date:	July 22, 2026
Track:	Java FSE Angular Track

1. Objective

The primary objective of this exercise is to implement an advanced, modern routing architecture inside the Student Course Portal. By integrating Lazy Loading across all major page components, we optimize the initial bundle size and application performance. We introduce parameterized routes for retrieving and rendering course details (Syllabus and Reviews nested tab components), bind ActivatedRoute query parameters to execute departmental lists filtering, and establish AuthGuard (CanActivate) security gates to block unauthorized profile pages access. Furthermore, we construct a CanDeactivate guard to prompt unsaved form modifications alerts, and implement a custom 404 Not Found fallback view.

2. Angular Concepts Covered

- **Lazy Loading (loadComponent):** Dynamically downloading component script bundles only when navigating to their respective routes, improving startup load speeds.
- **Route Parameters (paramMap):** Capturing dynamic URL path tokens (e.g. /courses/:id) to fetch and render specific resource records.
- **Query Parameters (queryParamMap):** Passing state filters through URL keys (e.g. /courses?department=Computer Science) to auto-initialize view filters.
- **Nested Child Routes (children):** Defining hierarchical routing nodes that load inside parent component sub-router outlets (e.g. Syllabus vs Reviews tabs).
- **Auth Guard (CanActivateFn):** Protecting private routes by running interceptor logic (e.g. checking simulated login session and redirecting unauthorized guests).
- **CanDeactivate Guard (CanDeactivateFn):** Protecting dirty input forms by prompting user confirmation checks before leaving the page.
- **Wildcard Routing (**):** Mapping unmatched paths to a dedicated fallback component (404 page).

3. App Routes Configuration (src/app/app.routes.ts)

```
export const routes: Routes = [
  { path: '', loadComponent: () => import('./pages/home/home').then(m => m.Home) },
  { path: 'courses', loadComponent: () => import('./pages/course-list/course-list').then(m => m.CourseList) },
  {
    path: 'courses/:id',
    loadComponent: () => import('./pages/course-detail/course-detail').then(m => m.CourseDetail),
    children: [
      { path: '', redirectTo: 'syllabus', pathMatch: 'full' },
      { path: 'syllabus', loadComponent: () => import('./pages/course-detail/syllabus/syllabus').then(m => m.CourseSyllabus) },
      { path: 'reviews', loadComponent: () => import('./pages/course-detail/reviews/reviews').then(m => m.CourseReviews) }
    ]
  },
  {
    path: 'profile',
    loadComponent: () => import('./pages/student-profile/student-profile').then(m => m.StudentProfile),
    canActivate: [AuthGuard],
    canDeactivate: [canDeactivateProfile]
  },
  { path: 'login', loadComponent: () => import('./pages/login/login').then(m => m.Login) },
  { path: 'not-found', loadComponent: () => import('./pages/not-found/not-found').then(m => m.NotFound) },
  { path: '**', redirectTo: 'not-found' }
];
```

4. Route Guards Implementation

CanActivate AuthGuard (src/app/services/auth.guard.ts)

```
export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);
  const notificationService = inject(NotificationService);

  if (authService.isAuthenticated()) { return true; }
  else {
    notificationService.show('Please log in to view the profile page.', 'warning');
    router.navigate(['/login']);
    return false;
  }
};
```

CanDeactivate Guard (src/app/services/deactivate.guard.ts)

```
export const canDeactivateProfile: CanDeactivateFn<StudentProfile> = (component) => {
  if (component.profileForm && component.profileForm.dirty && !component.showSuccess) {
    return confirm('You have unsaved changes. Do you really want to leave this page?');
  }
  return true;
};
```

5. Captured Screenshots & Verification

All routing views, param states, and guards were verified in the browser:

[Screenshot Placeholder]: Home page displaying new departmental shortcuts mapped to routing links with query parameters

(File not found at:

C:\Users\User\gemini\antigravity-ide\brain\fb0647-8016-412c-94ac-d4c721e05542\profile_h7_home_filters.png)

[Screenshot Placeholder]: Courses view auto-filtered to 'Computer Science' after navigating from the home shortcut card (File not found at: C:\Users\User\gemini\antigravity-ide\brain\fb0647-8016-412c-94ac-d4c721e05542\profile_h7_courses_filtered.png)

[Screenshot Placeholder]: Course details view (/courses/2) displaying dynamic course metadata and rendering nested Syllabus component (File not found at:

C:\Users\User\gemini\antigravity-ide\brain\fb0647-8016-412c-94ac-d4c721e05542\profile_h7_course_syllabus.png)

[Screenshot Placeholder]: Nested reviews route (/courses/2/reviews) displaying ratings and student reviews (File not found at:

C:\Users\User\gemini\antigravity-ide\brain\fb0647-8016-412c-94ac-d4c721e05542\profile_h7_course_reviews.png)

[Screenshot Placeholder]: Fallback wildcard routing redirecting unmatched URLs to a dedicated custom 404 NotFound page (File not found at: C:\Users\User\gemini\antigravity-ide\brain\fb0647-8016-412c-94ac-d4c721e05542\profile_h7_not_found.png)

[Screenshot Placeholder]: CanDeactivate guard intercepting page departure and prompting dirty form discard warnings (File not found at: C:\Users\User\gemini\antigravity-ide\brain\fb0647-8016-412c-94ac-d4c721e05542\profile_h7_deactivate_guard.png)

[Screenshot Placeholder]: AuthGuard blocking unauthorized access to /profile when logged out, redirecting to login page with alert toast (File not found at:

C:\Users\User\gemini\antigravity-ide\brain\fb0647-8016-412c-94ac-d4c721e05542\profile_h7_auth_redirect.png)

6. Unit Test Executions

All routing modifications and service components compiled cleanly. All unit tests successfully executed and passed in the Vitest test runner. The test output confirms passing specs across all modules.

7. Learning Outcomes & Conclusion

- ✓ Configured lazy loading on standalone page components to speed up startup load speeds.
- ✓ Captured dynamic path tokens using `ActivatedRoute ParamMap` subscriptions to load granular item records.
- ✓ Filtered dataset lists based on incoming query parameter tokens.
- ✓ Protected state paths using Auth guards and dirty-state `CanDeactivate` confirm overlays.
- ✓ Set up wildcard fallback routes to render dedicated custom error views.
- ✓ Resolved routing dependencies and providers inside spec testing harnesses.

Conclusion

Hands-On 7 successfully completes the navigation architecture of the Student Course Portal. The application incorporates modular lazy loading, route safety guards, nested child views, and search state query bindings, meeting all standard enterprise routing criteria.